Abstract

Diversity has been recognized as valuable in a number of tasks involving the selection of a subset of elements from a large data set. However, there are multiple definitions of diversity in the literature. In this paper, we show that these differences are not merely mathematical niceties: they significantly impact both what is accomplished and the cost of accomplishing it. We then argue for the use of one notion, coverage, as likely to be preferred in most situations. We support this argument through the study of a novel diversity problem in the context of LLMs.

Notes on the mix between Directional Queries and Diversity

Davide Martinenghi

October 2025

1 Introduction

Diversity is desired in many contexts. This desire is often reflected in a subset selection task: given a data set, we need to find a (usually small) subset that includes elements that meet or optimize some "quality" criterion, while at the same time meeting some diversity requirement. The quality criterion is usually measured by applying some scoring function to the attribute values of each tuple. A simplistic way to look at this is that the user has a particular need in mind but expresses it only in a limited way in the query presented to the system. How should the system respond so that the user's need is addressed? An example of this could be a product search where the user specifies some criteria, but has several additional unstated requirements or preferences.

The difficulty for the system is that it doesn't know precisely what the particular user desires, but it does know that users have diverse preferences. So, it tries to present a few diverse options in the hopes that the user will find something likeable within the first few choices presented. Which then begs the question of how to measure diversity.

As we discuss in Section **??**, there are two main families of measures, which we call *dispersion* and *coverage*. Within each family, there are further choices: Do we want to focus on the worst-case or the average case? Is it a global criterion applied to the returned set or an incremental criterion applied to each element as it is added? And so on. See excellent survey papers, such as [**?**], for details.

Intuitively, a dispersion measure wants data points that are far apart from each other. On the other hand, a coverage measure chooses data points that can "cover" or "represent" the full data set, in that there is a point in the selected subset not too far from every point in the full data set. Even though these two concepts are related, they can result in quite different choices. Furthermore, their mathematical properties are quite different.

When one speaks of diversity, the natural tendency is often to go towards dispersion, because it only considers the data points in the selected subset, whereas coverage requires an extrinsic notion of the set, or space, being covered. Yet, coverage is the more appropriate definition for many situations that seek diversity and has the important added benefit of computational tractability. We

present several short examples to illustrate this point before we dive into one novel in-depth study.

Large language models (LLMs) have shown remarkable capabilities in many data-related tasks in recent years. In particular, ...

2 Old Introduction

We discussed possible combinations of the techniques presented in [1] and [3].

2.1 Adding balance-based penalties to diversify neighbors

The first proposal regards a revised version of the basic algorithm presented in [3], whose goal was to determine a set of diverse points in a dataset with respect to a target (query) point q and a threshold angle θ.

The original idea roughly[1] consists of first selecting the point p_1 that is closest to the target q; this determines a first "cone of exclusion" C_1, which is the cone with apex q, axis directed along $\vec{qp}_1$ and aperture 2θ. The next point (p_2) in the result set is the closest to q that lies outside cone C_1. We proceed in this way by adding a cone of exclusion C_i for every added point p_i.

The alternative approach that we discussed has similar steps, but no longer uses cones. Instead, each time a point is added, the distance function is modified so as to take into account a sort of penalty due to the closeness to the already selected points. This penalty can be computed for every candidate point p' by appropriately weighting the angle $\angle pqp'$ formed with any already selected point p (and the target q). We observed that, at the i-th step, the penalty would consist of i components, which can be aggregated in several ways (e.g., average, sum, max, ...). Assuming that the initial distance function is, e.g., Euclidean distance (let's denote it $dist(\cdot,\cdot)$), the revised "distance" function $dist^{(i)}(p,q)$ (between a candidate point p and the target q) adopted at the i-th step ($i > 1$) may be expressed as follows:

$$dist^{(i)}(p,q) = (1-\lambda)dist(p,q) + \lambda c Agg_{j=1}^{i-1}(\pi - \angle pqp_j), \tag{1}$$

where Agg is an aggregate function, c is a multiplicative factor that might be needed to adjust scales between angles (in $[0,\pi]$) and distances (which could be a priori normalized in the $[0,1]$ interval). Note that the angles that we are aggregating upon are the supplementary angles of the $\angle pqp_j$'s because we want angles to be large, so we pay a penalty for small angles.

Possible instances of (1) are the following:

$$dist_{\min}^{(i)}(p,q) = (1-\lambda)dist(p,q) + \lambda\left(1 - \frac{\min_{j=1}^{i-1}\angle pqp_j}{\pi}\right), \tag{2}$$

[1]The actual strategy is slightly different, because the cones of exclusions are not only applied to the selected points, but also to the points θ-dominated by those (or so I understand).

$$dist^{(i)}_{sum}(p,q) = (1-\lambda)dist(p,q) + \lambda\left(1 - \frac{\sum_{j=1}^{i-1} \angle pqp_j}{\pi \cdot (i-1)}\right). \quad (3)$$

The algorithm described so far proceeds greedily to select the next point and does not need to be initialized with a threshold angle θ. It can be executed for the desired number of iterations so as to retrieve the target number of points (k) and it nicely falls back to the plain k-nearest neighbors problem when $\lambda = 0$.

We also wondered whether it was possible to express the problem solved by this algorithm in terms of a general objective function that needs to be optimized. A possible attempt:

Problem 2.1 *Find the set of k elements $p_1, \ldots, p_k$ in D that minimize the following objective function:*

$$(1-\lambda)\sum_{i=1}^{k} dist(p_i, q) + \lambda c \sum_{i=1}^{k} Agg_{j=1}^{i-1}(\pi - \angle p_i q p_j). \quad (4)$$

Simpler formulations may leave out constant additive terms, include the involved angles only once instead of twice ($Agg_{j>i}$), or just consider the most expensive angle contribution, so as to avoid the double level of aggregation (e.g., $\max_{j \neq i}(\pi - \angle p_i q p_j)$).[2]

Observation. The solution to Problem 2.1 and its simpler variants may not correspond to what is found by the greedy algorithm.

2.2 Adding diversity to directional queries

The second proposal regards the addition of the notion of diversity to directional queries. Directional queries are top-k queries equipped with a scoring function that combines a traditional utility-based component $\sum_{i=1}^{d} w_i t[i]$ with a "balance"-based component $\textsc{Dist}(t,\text{PL}(w))$ expressing the distance to the so-called "preference line" $\text{PL}(w)$. Both depend on a given weight vector w:

$$f_{dir}(t,w) = \beta \sum_{i=1}^{d} w_i t[i] + (1-\beta)\ \textsc{Dist}(t, \text{PL}(w)). \quad (5)$$

These scoring functions correspond to queries whose fronts are shaped like cones, roughly because we are willing to give up some utility in order to gain more balance (i.e., closeness to the preference line) in the results – the tip of such a cone has the same score as some other point closer to the origin (i.e., the target) but farther from the preference line. Although the very notion of balance seems to be in contrast with the idea of diversity, there might be natural scenarios in which one wishes to have both. One might for example indicate an "admissible" region of solutions expressed in terms of utility and balance, i.e.,

[2] But then, to have comparable terms, we may want to use max instead of $\sum$ for the *dist*-based part, too.

in terms of a cone; within that region, one may wish to find solutions that are as diversified as possible.

The problem at stake might be formulated in different ways. A possible problem formulation, slightly different from the description above, is as follows:

Problem 2.2 *Find the set of k elements $p_1, \ldots, p_k$ in D that minimizes the following objective function:*

$$(1-\lambda)\sum_{i=1}^{k} f_{dir}(p_i, w) + \lambda \max_{1\leq i,j\leq k}(1 - \textsc{Dist}(p_i, p_j)). \tag{6}$$

Here, I wrote $(1 - \textsc{Dist}(p_i, p_j))$, assuming distances are in $[0, 1]$ because we want to maximize the minimum distance, but for this function we are aiming at minimization. As usual, λ is a parameter for mixing the two components: the directional query score and the diversity score. Function (6) could be reformulated by removing constant additive terms.

Another formulation, closer to how the problem was described, could be to have a plain diversification problem (maximum minimum distance) with a constraint on the solutions:

Problem 2.3 *Find the set of k elements $p_1, \ldots, p_k$ in D that maximizes the following objective function:*

$$\begin{cases} \min_{1\leq i,j\leq k} \textsc{Dist}(p_i, p_j) \\ \textit{subject to } f_{dir}(p_i, w) \leq B, \end{cases} \tag{7}$$

where B is a budget characterizing the admissible region.

Algorithmically, Problem 2.3 could be attacked by using any of the existing solutions to diversification applied to only the points belonging to the admissible region.

3 Open issues

- Is there any algorithm better than brute force that can solve Problems 2.1 or 2.2 at optimality? Maybe something based on branch-and-bound (but what bounds could one introduce here?). One observation is that, as long as $n \gg k$, finding a subset of size k out of n elements is not prohibitively expensive, since there are roughly n^k sets, with k small, and computing the cost of each set takes roughly $O(k)$ or $O(k^2)$ time, depending on the kind of cost function. A brute-force solution should therefore be obtained with that cost. For a branch-and-bound solution, an upper bound can be computed by following the local-optimum choice for each of the k items to be added to the solution, whereas, at the i-th iteration (when $i-1$ items are already chosen), with $1 \leq i \leq k$, the cost c_i associated with the locally optimum i-th item also tells us that the lower bound cost for completing

the solution with the remaining $k - i + 1$ items is $c_i \cdot (k - i + 1)$. We may also want to keep partial solutions in a sort of lexicographic order so as to avoid computing permutations of the same set. N.B. The cost function should then have some sort of additivity property, which probably applies to the scenarios we considered so far.

- Is there a strong motivation for at least one of the two Problems 2.1 and 2.2? What convincing, natural scenarios are there? What cases of non-linear scoring functions would be useful in natural problems, besides directional scores? Examples that come to mind include livableness value of neighborhoods (which does not vary linearly with the geography) and hard constraints on, e.g., budgets (say you are allowed to spend up to 200$ but no more, so 201$ corresponds to a ∞ cost, i.e., with a highly non-linear transformation).

- In the formulation of Problem 2.1, there might some hidden normalization factor that was implicitly assumed and that might play a role especially in higher dimensionalities. That should be somehow taken into account, without overburdening the user with too many parameters.

- Regret queries are somehow tackling a similar problem, i.e., finding top, diversified items. We should compare to them and justify the need of another approach. One of the aspects that might be used in that respect is the fact that regret queries, at least in their initial formulations, only consider linear scoring functions, thereby failing to meet the requirements of balance, which are inherently non-linear.

- Prove that the greedy approach does not find the optimal solution (probably doable already with $k = 2$, since the optimal solution may not include the closest point, while the greedy algorithm does.

- Think of a dynamic programming approach (unlikely) or prove that it is not possible (more likely).

4 Examples

1. Consider Investment Portfolio Selection.

   Top-k Constraint: A fund manager or algorithmic system must select a small, fixed number (k) of assets (stocks, bonds, etc.) to include in a focused portfolio.

   Non-linear Utility Function: defined by a risk-adjusted return metric, which is inherently non-linear. The utility of the entire portfolio is non-linear because it depends on the covariance (how asset prices move relative to each other) among the selected assets.

   Diversification: The primary goal here is risk mitigation. The utility is maximized by selecting a diverse set of assets (e.g., stocks from different

industries, different geographies, or different asset classes like real estate and commodities). A lack of diversification (high similarity) means higher overall risk for the portfolio, which directly and non-linearly reduces the risk-adjusted utility score.

$$U = \mu_p - \lambda \cdot \sigma_p^2$$

μ_p (Portfolio Expected Return) is a linear function of the individual asset returns and their weights.

σ_p^2 (Portfolio Variance/Risk) is the non-linear component.

So here the non-linear component is only set-wise.

2. Housing: perception of quality of houses may not vary linearly with the size or with the commute time...

3. Drug Candidate Selection

   - The utility of an individual drug candidate is based on attributes like efficacy and toxicity, and is highly non-linear due to biological thresholds.
     - Efficacy/Potency: The utility gained from increasing a drug's potency is often sigmoidal (S-shaped). Below a certain concentration threshold, the drug has almost no effect (zero utility). Above a slightly higher concentration, the effect plateaus (diminishing marginal utility). The most critical region is the steep, non-linear climb between the minimum effective dose and the maximum useful dose.
     - Toxicity/Safety: The disutility (negative satisfaction) of toxicity is highly non-linear. A small increase in a toxicity marker above a regulatory threshold can cause the utility to drop instantly to zero, regardless of efficacy.
   - Diversity: The top-k compounds must be diverse in:
     - Molecular Structure: If all top compounds share the same scaffold, one biological failure mechanism could eliminate the entire pipeline.
     - Mechanism of Action: How the drug interacts with the disease target. Candidates should ideally hit the target through different biochemical pathways.

5 General formulation and approaches

The problem can be generally expressed as the maximization of a function such as

$$F(S) = \lambda \sum_{i \in S} u_i + (1 - \lambda) Div(S)$$

where u_i is the utility of the i-th point in the selected set S and $Div(S)$ is the diversity score of the set (for instance $Div(S) = \min_{i \neq j \in S} dist(i, j)$.

5.1 Overview of the cases

- if $\lambda = 1$, it's a top-k by utility, which can be solved, e.g., by a simple heap-based algorithm in $O(n \log k)$;
- if $\lambda = 0$, it's a k-dispersion max-min problem, which is NP-hard in general [2];
- for $0 < \lambda < 1$ the problem generally remains NP-hard.

5.2 Exact approaches

For an exact solution in the general case, we can formulate the problem as a MILP, where we represent the choice of k points through binary variables $x_i \in \{0, 1\}$ such that $\sum_i x_i = k$. The objective is the following:

$$\max \lambda \sum_i u_i x_i + (1 - \lambda)\delta$$

and the constraints are

$$\sum_i x_i = k$$

$$\delta \leq dist(i, j) + M \cdot (2 - x_i - x_j), \text{ for every } i < j$$

where δ is a real variable representing the minimum pairwise distance among selected points (so $Div(S) = \delta$), and M is a big enough constant so that the last constraint is effective only when both points are chosen (one can chose, e.g., $M = \max dist(i, j)$).

The number of constraints is high (quadratic in n), so this is not feasible for large sets.

5.3 Greedy approaches

- Greedy marginal gain: start with $S = \emptyset$ and add each time the point p maximizing the marginal objective, until $|S| = k$:

  $$p = \arg\max_{i \notin S} \lambda u_i + (1 - \lambda)(Div(S \cup \{i\}) - Div(S))$$

  The result can be computed in $O(kn)$ time.

- Cluster then pick: determine k clusters via a standard clustering algorithm (k-means, etc.) and pick the point with the best utility within each cluster.

5.4 Guarantees

Consider a different notion of diversity, called *facility-location*, expressed in terms of similarity between points:

$$g(S) = \sum_{i \in D} \max_{j \in S} w_{ij} \tag{8}$$

where w_{ij} is similarity (or closeness, etc.) between items i and j. This notion captures the fact that each point in the dataset D is covered by the best (most similar) center $j \in S$ and we want to maximize this representativeness of the selected set S with respect to the entire dataset D. Such a function g is monotone submodular and, when combined with a modular (linear) function such as cumulative utility, the overall function is still *monotone submodular*.

Monotone here means that $A \subseteq B$ entails $g(A) \leq g(B)$. Submodularity is the property of "diminishing returns" (the larger set gets smaller marginal gains), i.e., $A \subseteq B$ entails that, for every $x \notin B$, $g(A \cup \{x\}) - g(A) \geq g(B \cup \{x\}) - g(B)$. It is plain that the function $g_v(S) = \max_{j \in S} w_{vj}$ is submodular; also, the nonnegative sum of submodular functions is submodular, so g is submodular.

Observation. The overall objective function at hand

$$F(S) = \lambda \sum_{i \in S} u_i + (1 - \lambda) g(S)$$

is monotone submodular (and nonnegative).

For monotone submodular functions it is well known [4] that the standard greedy algorithm provides a solution S_{greedy} that approximates the optimal k-set S^* well:

$$F(S_{greedy}) \geq \left(1 - \frac{1}{e}\right) F(S^*),$$

where $(1 - 1/e) \approx 0.632$.

This formulation is suitable for recommendations, summarization and sensor placement.

Below, a few scenarios where F might be the right objective function to optimize:

- summarization: find a representative set of size k out of a large dataset D, where the points has importance (u_i) and facility-location ensures that every point in D is well served by at least one representative $i \in S$. Examples: representative images (diverse but visually important), core-set for large ML training datasets (high-value examples and broad coverage), summarization of customer profiles into representative personas weighted by business value.

- recommendation systems: here, facility-location ensures the "covering" of the space. Examples in movies, news, e-commerce, etc.

- sensor or facility placement: some sites have higher expected yield (utility) and every point in the region must be covered by at least one chosen location (facility-location).
- influence maximization: pick k influencers in a social network. Utility = influence, facility-location = each node must be "covered" by the most influential reachable influencer.
- balanced portfolio: expected return (utility) and coverage of different areas (e.g., industry sectors).
- fair balanced selection: encode affinity to demographic groups with w_{ij} so that facility-location ensure coverage of all groups.

6 Exact branch-and-bound solution

We refer to the facility-location formulation of diversity given in Equation (8), where we have a set D of n items, each equipped with a nonnegative utility u_i. For all $i, j \in D$ we are given nonnegative similarities $w_{ij} \geq 0$. For any subset $S \subseteq D$ the facility-location coverage is

$$g(S) = \sum_{i \in D} m_i(S), \qquad m_i(S) := \max_{j \in S} w_{ij},$$

with $m_i(\emptyset) = 0$. Given $0 \leq \lambda \leq 1$, we want to maximize the objective

$$F(S) := \lambda \sum_{i \in S} u_i + (1 - \lambda) g(S).$$

In particular, for a given target size k, we want to compute

$$S^* = \text{argmax}_{S \subseteq D,\ |S|=k}\, F(S).$$

The nodes in the search are partial sets S_p of elements, with $|S_p| \leq k$. At each node we maintain:

- $F(S_p)$ (value of partial set),
- $m_i(S_p)$ for all $i \in D$ (current coverage of points),
- marginal gains $\Delta(j \mid S_p) = F(S_p \cup \{j\}) - F(S_p)$ for candidates $j \notin S_p$.

We use two bounds to prune the space:

1. Lower bound $\text{LB}(S_p)$ obtained through any feasible completion.
2. Upper bound $\text{UB}(S_p)$ on the best objective obtainable by adding $t = k - |S_p|$ elements to S_p; for this, we can exploit submodularity:

$$\text{UB}(S_p) = F(S_p) + \sum_{j \in R_{(1:t)}} \Delta(j \mid S_p), \tag{9}$$

where $R_{(1:t)}$ are the t largest marginals among candidates.

Algorithm 1 Exact Branch-and-Bound for Utility + Facility-Location

Require: Dataset D, Utilities u_i, similarity matrix w_{ij}, parameter λ, size k

Initialize priority queue Q with root node containing $\emptyset$
BestSolution := $\emptyset$; BestValue := $-\infty$ *// or initialize with a greedy LB*
while Q not empty **do**
    Pop node N with highest upper bound UB(N)
    if UB(N) $\leq$ BestValue **then**
        prune node
        continue
    end if
    if $|N.S| = k$ **then**
        if $F(N.S) >$ BestValue **then**
            BestSolution := $N.S$; BestValue := $F(N.S)$
        end if
        continue
    end if
    for each $e \in D \setminus N.S$ **do** *// sorted by marginal gain, descending*
        Create child node N' with $N'.S = N.S \cup \{e\}$
        if UB(N') $>$ BestValue **then**
            Push N' into Q
        end if
    end for
end while
return BestSolution

At each iteration, the branch-and-bound policy prunes a partial solution S_p if $\mathrm{UB}(S_p) \leq$ current-best; otherwise it branches by creating children $S_p \cup \{j\}$ in a promising order (descending marginals). The pseudocode corresponding to this idea is shown in Algorithm 1.

To prove correctness of Equation (9), take any partial set S_p and any completion $T \subseteq D \setminus S_p$ with $|T| = t$. Since submodularity and monotonicity imply the "diminishing returns" property, we have:

$$\begin{aligned} F(S_p \cup T) - F(S_p) &= \sum_{i=1}^{t} \Big(F\big(S_p \cup \{t_1, \ldots, t_i\}\big) - F\big(S_p \cup \{t_1, \ldots, t_{i-1}\}\big) \Big) \\ &\leq \sum_{i=1}^{t} \Big(F(S_p \cup \{t_i\}) - F(S_p) \Big) = \sum_{j \in T} \Delta(j \mid S_p). \end{aligned}$$

Hence the maximum incremental gain achievable by adding t elements is at most the sum of the t largest single-element marginals $\Delta(\cdot \mid S_p)$, as in UB (9).

In the pseudocode, a greedy lower bound may be used as the initial BestSolution, obtained by repeatedly picking the element with largest current marginal until k elements are selected. Also, as a variant of the algorithm, a greedy lower bound might even be computed for each partial set to enforce earlier pruning in case the lower bound increases.

6.1 Observations on asymptotic complexity

The search tree explores combinations of size at most k out of n items. The number of possible leaves is $\binom{n}{k}$ and the number of possible nodes is $\sum_{i=0}^{k} \binom{n}{i}$, so worst-case time is exponential in k (polynomial in n for fixed k).

At each node, we need to do the following

- compute exact marginals Δ_j for all candidates $j \in D \setminus S_p$. This requires computing, for the diversity part, $\sum_{i \in D} \max(0, w_i j - m_i)$, which is $O(n)$. So overall $O(n^2)$.
- select top-t marginals (to form UB, which is $O(n \log k)$ using a heap.
- update coverage array m_i when adding one element ($m_i := \max(m_i, w_{ij})$), which over $i \in D$ takes $O(n)$.
- using priority queue requires $O(\log |Q|)$, which is $O(\log |N_{nodes}|)$, where N_{nodes} is the number of nodes actually explored by B&B.

The heaviest component seems to be the cost of computing exact marginals $O(n^2)$, issuing an overall cost of $O\left(\sum_{i=0}^{k} \binom{n}{i} \cdot n^2\right)$.

7 Experiments

Inspired by [5], we aim to define a pre-LLM context selection configuration and to study whether replacing a diversity-oriented objective with an explicit

coverage objective improves the quality of the resulting context, compared to a relevance-only baseline.

Given a query and a set of retrieved textual candidates, the task is to select a subset of text segments under a fixed token budget. In the proposed pipeline, each example is first associated with a pool of candidate texts extracted from one or more source documents. Depending on the dataset, candidates may correspond to document chunks (e.g., in TriviaQA [3]) or to shorter textual units such as sentences (e.g., in TREC-style benchmarks [4]). When documents are available, they are segmented into overlapping token-based chunks; each of these candidates is then embedded using a dense encoder, and its relevance to the query is estimated via cosine similarity.

Two selection strategies are compared. The baseline selects candidates by descending relevance until the global token budget is exhausted. In the coverage-based strategy, instead, each candidate contributes to both relevance and marginal coverage, measured as the additional semantic space it covers relative to already selected items. The trade-off between relevance and coverage is controlled by a weighted combination of the two objectives.

The effectiveness of coverage-based selection depends critically on the structure of the candidate pool. For this reason, several parameters governing candidate generation, budget constraints, and selection dynamics are exposed and systematically varied to assess under which conditions coverage-aware selection provides measurable benefits across different datasets.

- **Dataset choice**: datasets with higher evidence redundancy may mask coverage benefits, while datasets with fewer but more distinct sources may better highlight trade-offs.
- **Coverage**: current experiments use Equation (8) to calculate coverage, but other approaches can be used, such as defining groups of documents according to their metadata.
- **Coverage weight (λ)**: controls the trade-off between relevance (embedding similarity) and coverage gain in the selection objective. Lower λ increases the influence of coverage but may decrease relevance.
- **Embedding model**: different encoders affect similarity distributions and redundancy across chunks, indirectly influencing coverage gains.
- **Chunk size and overlap**: we don't rate entire documents, but we usually divide the documents into chunks, and we allow for a small overlap among chunks. Larger chunks increase answer redundancy within a single source, while smaller chunks and higher overlap increase the number of competing candidates, potentially making coverage more effective.

[3] https://huggingface.co/datasets/mandarjoshi/trivia_qa

[4] https://microsoft.github.io/msmarco/TREC-Deep-Learning#passage-ranking-dataset

- **Context budget**: total token budget for the concatenation of all selected chunks. A smaller budget forces stronger competition among chunks, which can amplify the differences in results for different ranking approaches.
- **Maximum number of candidates**: limits the size of the candidate pool considered by the selection algorithm. If the pool is truncated too aggressively, coverage effects may be suppressed.
- **Maximum documents per example**: controls how many source documents contribute to the candidates pool. Increasing this value introduces additional source diversity and reduces answer redundancy, creating more favorable conditions for coverage-based selection.

Starting from a default configuration inspired by [5], we modify a single parameter at a time to measure its impact on the results

7.1 Effect of the relevance-coverage trade-off

The first parameter explored is the relevance-coverage trade-off weight, denoted by λ. In the current implementation, candidate selection is formulated as a BUDGETED FACILITY-LOCATION PROBLEM, where each candidate chunk is scored as a weighted combination of its relevance to the query (estimated via embedding similarity) and its additional COVERAGE with respect to the already selected candidates.

Experiments were conducted on the TRIVIAQA dataset, using the default configuration inspired by [5], where $\lambda = 0.7$. Starting from this reference value, we evaluated a range of settings by decreasing λ from 0.9 down to 0.4, with a step size of 0.1, adjusting the complementary coverage weight accordingly at each run.

Across this range of values, no substantial differences were observed between the relevance-only baseline and the coverage-aware selection strategy. In particular, both answer recall (measured as answer string match within the selected context) and the coverage metric itself remained very close between the two approaches. This suggests that, under the current experimental configuration, the candidate pool may be too homogeneous or redundant for the coverage objective to induce a meaningful change in the selected context.

For this reason, before further refining the choice of λ, the next experimental phase focuses on parameters that directly affect candidate generation and competition.

7.2 Effect of context budget and chunking configuration

To test whether the lack of separation between relevance-only and coverage-aware selection is due to limited competition in the candidate pool, we next varied parameters that directly control (i) how many candidates are generated and (ii) how strongly they compete under a fixed context budget. Concretely,

Setting	$T_{\max}$	λ	ch	st	hit_b	hit_c	Δhit	Δfac
Default budget, varying λ	2048	0.7	256	128	0.9056	0.9069	+0.0013	+0.0020
	2048	0.6	256	128	0.9056	0.9072	+0.0016	+0.0030
	2048	0.5	256	128	0.9056	0.9077	+0.0020	+0.0043
	2048	0.4	256	128	0.9056	0.9081	+0.0025	+0.0060
Tighter context budget	512	0.7	256	128	0.8397	0.8456	+0.0059	+0.0034
	256	0.7	256	64	0.7646	0.7688	+0.0043	+0.0029
Alternative chunking/overlap								
($T_{\max} = 512$)	512	0.7	128	64	0.8398	0.8434	+0.0036	+0.0028
	512	0.7	128	32	0.8522	0.8547	+0.0025	+0.0020
	512	0.7	64	32	0.8187	0.8205	+0.0018	+0.0018
	512	0.7	64	16	0.8375	0.8386	+0.0011	+0.0013

Table 1: Summary of completed TriviaQA runs. Here λ denotes the relevance weight (with coverage weight $1 - \lambda$); `ch`=`chunk_tokens` and `st`=`stride`. We report baseline vs. coverage hit rates and the deltas Δ = coverage − baseline for hit rate and facility coverage (cosine-based).

we explored smaller context budgets $T_{\max}$ (from 2048 down to 512 and 256 tokens) and alternative chunking/overlap settings by changing `chunk_tokens` and `stride`. Across these runs, the absolute hit rates decreased substantially when the budget became tighter (as expected), but the *gap* between the baseline and the coverage-aware strategy remained modest overall. The largest improvements in the recall observed so far are on the order of a few tenths of a percentage point (i.e., Δ`hit` ≤ 0.006, suggesting that (at least under the current embedding model and facility-location formulation) coverage changes the selected context only mildly. Importantly, some configurations also change the number of valid examples (i.e., examples with at least 10 extracted candidates), indicating that chunking and budget constraints influence not only the ranking dynamics but also the effective evaluation set.

Table 1 summarizes the experiments completed so far on TriviaQA. We report both absolute metrics and the key deltas Δ = coverage − baseline for hit rate and facility coverage. In the next stage (currently running), we further increase candidate diversity by varying `max_docs` and `max_cands`, while keeping fixed the configuration that yielded the largest Δ`hit` observed so far.

7.3 Effect of candidate pool size

After analyzing the impact of the relevance-coverage trade-off parameter λ, the next experimental phase focuses on properties of the candidate pool itself. In particular, we investigate how the size and composition of the pool affect the potential benefits of coverage-aware selection.

Two parameters are especially relevant in this regard:

- **Maximum documents per example** (`max_docs`): determines how many

source documents contribute to the candidate set. Increasing this value can introduce additional topical diversity and reduce redundancy across sources, potentially making coverage-based selection more impactful.

- **Maximum number of candidates** (`max_cands`): limits the total number of chunks considered by the selection algorithm. If this limit is too restrictive, most candidates may originate from a small number of highly ranked documents, leaving little room for coverage to influence the final selection.

All experiments in this phase are conducted using the best configuration identified so far in terms of baseline–coverage gap, namely:

$$T_{\max} = 512, \quad \lambda = 0.7, \quad \texttt{chunk\_tokens} = 256, \quad \texttt{stride} = 128.$$

These settings are kept fixed while varying `max_docs` and `max_cands`.

The rationale behind these experiments is the following. Increasing `max_docs` should enlarge the diversity of information sources available to the algorithm, while increasing `max_cands` allows more candidate chunks from those documents to compete for inclusion. Both changes are expected to create more favorable conditions for coverage-aware selection by reducing the dominance of a few highly redundant candidates.

In addition to recall and facility-location coverage, we introduce a diagnostic metric based on the Jaccard similarity between the sets of chunks selected by the two strategies.

Let S_{base} and S_{cov} denote the sets of chunk indices selected by the relevance-only baseline and by the coverage-aware method, respectively. We compute

$$J(S_{\text{base}}, S_{\text{cov}}) = \frac{|S_{\text{base}} \cap S_{\text{cov}}|}{|S_{\text{base}} \cup S_{\text{cov}}|}.$$

Values close to 1 indicate that the coverage-aware strategy largely reproduces the baseline selection, while lower values indicate that the two methods choose substantially different chunks under the same budget.

Table 2 summarizes the results. Despite substantial increases in candidate pool size (up to `max_docs`=30 and `max_cands`=600), the improvement in hit rate remains constant at $\Delta \approx 0.0059$, and facility-location coverage shows only minimal variation. Most importantly, the Jaccard similarity between the two selections remains extremely high (≈ 0.86 for selected chunks and ≈ 0.95 for domains), indicating that the coverage-aware strategy is, in practice, selecting almost exactly the same content as the relevance-only baseline.

These findings suggest that, under the current configuration, increasing the pool size does not create sufficient competition among candidates to induce meaningfully different selections. The behavior appears to be largely dominated by the top-relevance chunks, which remain stable even when additional candidates are made available.

Setting	max_docs	max_cands	Hit_{base}	Hit_{cov}	Δ	Jaccard_{chunks}	$\text{Jaccard}_{domains}$
docs20_c200	20	200	0.8397	0.8456	0.0059	0.8584	0.9456
docs12_c400	12	400	0.8397	0.8456	0.0059	0.8584	0.9456
docs20_c400	20	400	0.8397	0.8456	0.0059	0.8584	0.9456
docs30_c600	30	600	0.8397	0.8456	0.0059	0.8584	0.9456

Table 2: Impact of candidate pool size on TriviaQA. High Jaccard values indicate that baseline and coverage-aware strategies select nearly identical sets of chunks and domains, even when the pool size is substantially increased.

Setting	PCA	Hit_{base}	Hit_{cov}	Δ	Jaccard_{chunks}	$\text{Jaccard}_{domains}$	FacCov_{base}	FacCov_{cov}
v2 (no PCA)	no	0.8397	0.8456	0.0059	0.8584	0.9456	0.8736	0.8770
v2_pca128	yes	0.8190	0.8222	0.0032	0.9576	0.9829	0.1875	0.1905

Setting	GoldConcat_{base}	GoldConcat_{cov}	Gold/Chunk_{base}	Gold/Chunk_{cov}
v2 (no PCA)	1.4559	1.4718	1.1361	1.1350
v2_pca128	1.3992	1.4061	1.0524	1.0506

Table 3: Comparison of two TriviaQA runs with identical settings (see common setup above), differing only in the use of per-example PCA (128 dims). $\Delta = \text{Hit}_{cov} - \text{Hit}_{base}$.

7.4 Adding new metrics in the best configuration

Table 3 shows the results for the best configuration found on TriviaQA, and adds two additional metrics:

GoldConcat metrics. For a text T (the concatenation of all selected chunks), counts the set of unique gold aliases found in T.

Gold/Chunk metrics. For each selected chunk counts the unique gold aliases present in that single chunk.

The two runs share the same TriviaQA setup and selection hyperparameters (Tmax=512, α=0.7, bonus=0.3, max_docs=20, max_cands=200), differing only by enabling per-example PCA (128 dims). Without PCA, coverage improves Hit from 0.8397 to 0.8456 ($\Delta = +0.0059$), increases domain diversity (1.3971 $\rightarrow$ 1.4229), and slightly improves facility coverage (0.8736 $\rightarrow$ 0.8770). With PCA128, both Hit scores drop (0.8190 $\rightarrow$ 0.8222, $\Delta = +0.0032$) and facility coverage collapses (0.1875 $\rightarrow$ 0.1905), while Jaccard overlaps increase markedly (chunks: 0.8584 $\rightarrow$ 0.9576; domains: 0.9456 $\rightarrow$ 0.9829), indicating that baseline and coverage select much more similar chunk/domain sets under PCA. Gold-alias metrics are also lower with PCA (GoldConcat_{cov}: 1.4718 $\rightarrow$ 1.4061; Gold/Chunk_{cov}: 1.1350 $\rightarrow$ 1.0506), consistent with weaker retrieval signals in the PCA-transformed space.

Example of an output

```
QUESTION:
Which oil scandal hit the US in 1924?

GOLD ANSWERS (aliases, truncated):
teapot scandal; teapot dome scandal; teapot dome; tea pot dome scandal; teapot
     dome affair; elk hills oil field; tea pot dome; tea pot dome scandel

BASELINE SELECTED CHUNKS (n=2):
  [1] domain=britannica.com | unique_gold_in_chunk=2
      teapot dome scandal | united states history | britannica. com ... (chunk
           truncated)
  [2] domain=freedom-school.com | unique_gold_in_chunk=2
      teapot dome scandal juggernaut. this 1924 cartoon shows ... (chunk
           truncated)

COVERAGE SELECTED CHUNKS (n=2):
  [1] domain=britannica.com | unique_gold_in_chunk=2
      teapot dome scandal | united states history | britannica. com ... (chunk
           truncated)
  [2] domain=spartacus-educational.com | unique_gold_in_chunk=1
      naval reserves... hearings on the teapot dome oil lease began... (chunk
           truncated)
```

7.5 TREC-DL experiments (BM25 candidates + embedding-based selection).

We replicated the pre-LLM selection analysis on the TREC Deep Learning passage retrieval benchmarks. For each query q, we need to retreive a pool of candidate passages and the dataset provides a human-labelled set of passages for each query. As before, given a token budget $T_{\max}$ and per-candidate token lengths ℓ_i (computed via tokenizer encoding), we compare two selection strategies: (i) a *baseline* that greedily selects the highest-reward candidates under the budget, and (ii) a *coverage-aware* greedy strategy that trades off reward and marginal coverage gain, $\text{score}(i) = \alpha\, r_i + \beta\, \Delta\text{cov}(i)$. Selected passages are finally *re-ranked by reward* (descending r_i) and evaluated with TREC scores.

In addition to the previous metrics we compute additional metrics using the TREC scores (qrels).

Recall@10 tells us how completely the system retrieves the relevant material for a query: it is the fraction of all relevant passages (as defined by the qrels) that appear somewhere in the top 10 retrieved results. A higher Recall@10 means that more of the truly relevant passages are successfully captured within the limited top-10 list, while a lower value indicates that many relevant passages are missing from the retrieved set.

nDCG@10 (normalized Discounted Cumulative Gain at 10) evaluates not only *whether* relevant passages are retrieved, but also *how well they are ranked*. It gives more credit when highly relevant passages appear near the top of the list

TriviaQA, chunk=512, stride=256 $T_{\max} = 10000$						
Setting	Method	Hit	Domains	FacCov	GoldConcat	GC/Jaccard
$\alpha = 0.70,\ \beta = 0.30$	Baseline	0.944	3.692	0.966	2.614	–
	Coverage	0.944	3.752	0.972	2.610	$J(B, C) = 0.836$
	MMR	0.945	3.697	0.969	2.418	Gold/Chunk=0.868
$\alpha = 0.95,\ \beta = 0.05$	Baseline	0.944	3.692	0.966	2.614	–
	Coverage	0.945	3.732	0.968	2.619	$J(B, C) = 0.961$
	MMR	0.944	3.696	0.966	2.421	Gold/Chunk=0.892

Table 4: TriviaQA with large context budget.

TriviaQA, chunk=512, stride=256 $T_{\max} = 2000$						
Setting	Method	Hit	Domains	FacCov	GoldConcat	GC/Jaccard
$\alpha = 0.70,\ \beta = 0.30$	Baseline	0.913	2.266	0.884	2.266	–
	Coverage	0.906	2.335	0.899	2.335	$J(B, C) = 0.393$
	MMR	0.916	2.251	0.887	1.890	Gold/Chunk=1.107
$\alpha = 0.95,\ \beta = 0.05$	Baseline	0.913	2.266	0.884	2.266	–
	Coverage	0.915	2.326	0.889	2.326	$J(B, C) = 0.780$
	MMR	0.914	2.266	0.884	1.904	Gold/Chunk=1.190

Table 5: TriviaQA results with smaller context budget.

and progressively less credit as relevant passages appear further down (because users and downstream models pay more attention to early ranks). The score is normalized by the best possible ranking for that query, so values are comparable across queries. A higher nDCG@10 therefore indicates both strong relevance and good ordering of the top-10 results.

Best observed improvement On `msmarco-passage/trec-dl-2020/judged` (54 queries) with $T_{\max} = 512$, $N = 200$, $\alpha = 0.7$, `bonus`= 0.3, the coverage-aware strategy slightly improves nDCG@10 from 0.5662 to 0.5681 ($\Delta = +0.0020$) and Recall@10 from 0.1857 to 0.1871 ($\Delta = +0.0014$), while also increasing facility coverage from 0.8581 to 0.8608. Jaccard is 0.88.

References

[1] Paolo Ciaccia and Davide Martinenghi. Directional queries: Making top-k queries more effective in discovering relevant results. *Proc. of ACM Management of Data*, 2(6):232:1–232:26, 2024.

Setting	Method	DocRec	FactRec	FacCov	AvgCos	J(Sim,·)
$w = 100$	Sim	0.830	0.823	0.641	0.424	–
	MMR	0.836	0.816	0.672	0.358	0.683
	gMMR	0.852	0.839	0.659	0.377	0.787
	Fac	0.766	0.740	0.724	0.336	0.527
$w = 200$	Sim	0.854	0.856	0.700	0.403	–
	MMR	0.848	0.848	0.723	0.352	0.736
	gMMR	0.870	0.871	0.713	0.367	0.827
	Fac	0.842	0.843	0.740	0.351	0.695

Table 6: HotpotQA distractor validation results. Chunk size $w \in \{100, 200\}$ words, $K = 5$.

TriviaQA (rc, validation), chunk=512, stride=256
$T_{\max} = 10000$

Setting	Hit_B	Hit_C	Hit_M	Domains_B	$J(B, C)$	$J(B, M)$	$J(C, M)$
$\alpha = 0.70, \beta = 0.30$	0.944	0.944	0.945	3.692	0.836	0.874	0.848
$\alpha = 0.95, \beta = 0.05$	0.944	0.945	0.944	3.692	0.961	0.979	0.967

Table 7: Overlap between retrieval sets (chunk-level). Jaccard is computed on the sets of selected chunk indices under the same $T_{\max}$.

[2] Erhan Erkut. The discrete p-dispersion problem. *European Journal of Operational Research*, 46(1):48–60, 1990.

[3] Qi Guo, H. V. Jagadish, Anthony Kum Hoe Tung, and Yuxin Zheng. Finding diverse neighbors in high dimensional space. In *34th IEEE International Conference on Data Engineering, ICDE 2018, Paris, France, April 16-19, 2018*, pages 905–916. IEEE Computer Society, 2018.

[4] George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. An analysis of approximations for maximizing submodular set functions–I. *Mathematical Programming*, 14(3):265–294, 1978.

[5] Zhichao Wang, Bin Bi, Yanqi Luo, Sitaram Asur, and Claire Na Cheng. Diversity enhances an llm's performance in rag and long-context task, 2025.